

管理应用。一方面，操作系统为应用提供了各种不同层次、不同功能的接口以提供不同类型的服务，如访问控制、应用间交互等。通过这些服务，操作系统将应用从繁杂的资源管理等系统工作中解放出来，使得 hello 应用可以通过少量的代码来实现屏幕显示的复杂功能。另一方面，操作系统也负责对应用的生命周期进行管理，包括应用的加载、启动、切换、调度、销毁等。通过对应用的管理，操作系统能够从全局角度进行资源分配，从而保证应用间的公平性、性能与安全的隔离性，防止和限制少数恶意应用对系统整体产生的影响。

由于操作系统的以上两个主要职责是分别面向硬件和应用的，因此当硬件和应用发生改变时，操作系统通常也需要随之进行调整——这正是当前的情况。一方面，当前的硬件种类越来越多。例如，智能驾驶汽车可包含通用处理器、人工智能加速器、高带宽内存、传感器、高清摄像头、毫米波雷达、激光雷达等各种硬件，操作系统需要对这些新的硬件进行管理与抽象，以便智能驾驶应用高效、方便地利用这些硬件资源。操作系统还需要支持这些硬件设备之间的相互协作，例如高清摄像头拍摄的图像需要传递给人工智能加速器进行处理。另一方面，应用的需求也越来越多样化。例如，控制类应用对时延的要求越来越苛刻，如飞行控制与智能驾驶应用需要操作系统提供实时性保障；移动应用则对低功耗的需求越来越高，如语音唤醒功能既要提供快速响应又要保持极低功耗。这些硬件和应用的新变化与新需求驱动着操作系统不断演进。

早期操作系统（如 UNIX）仅仅是操作系统内核加上一个简单的 shell。内核包含了操作系统的所有功能，通过系统调用的方式对外提供服务。shell 则以命令行的形式实现人机交互，利用操作系统的接口提供应用程序启动、切换、关闭等简单功能，如本章运行 hello 的命令行界面。

随着硬件种类和应用需求变得越来越丰富，大量共性功能沉淀到操作系统中，使操作系统的内涵与外延不断扩大。为了在稳定性与灵活性之间进行更好的权衡，操作系统在演进的过程中产生了一种新的形态：操作系统内核与操作系统框架的组合。操作系统内核通常用于实现通用的、相对稳定的功能；操作系统框架则更为灵活，可面向不同应用场景进行适配，提供更有针对性的功能。随着功能的逐步增多，操作系统框架可以进一步分为系统服务与应用框架，前者实现对操作系统内核抽象与管理能力的扩展，后者基于系统服务实现对应用开发与运行所需共性功能的增强。图 1.1 左侧展示了现代操作系统的简明结构：操作系统内核负责对硬件资源的管理与抽象，为操作系统框架提供基础的系统服务；系统服务则基于操作系统内核提供的功能进行进一步抽象与封装，从而方便上层应用框架的设计与开发；应用框架基于系统服务提供的功能，结合应用领域的特点提供应用开发与运行的共性功能，从而方便开发者开发与管理领域应用。图 1.1 右侧展示了 Android 操作系统的结构：Linux 内核提供对硬件的抽象与管理，Android 系统服务进一步扩展硬件能力，Android 应用框架为上层提供 Android 应用的开发与执行环境。

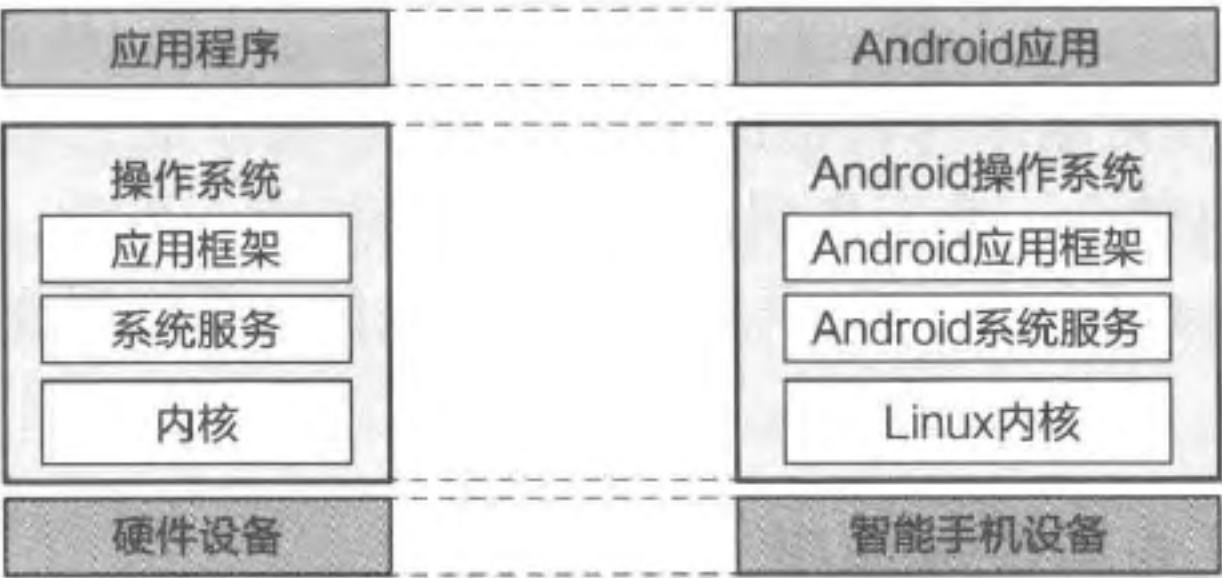


图 1.1 操作系统的简明结构

接下来，我们将沿着时间线，梳理一下操作系统的发展历史。

操作系统与应用

如果说操作系统的作用是应用管理和硬件抽象，那么如果只有一个应用，该应用直接控制硬件，是否还需要操作系统？如果应用直接和硬件打交道，直接操作和管理硬件资源，那么可以不需要操作系统。这么做的优点在于可减少由于操作系统对硬件的抽象而带来的性能损失，理论上可以 100% 发挥硬件性能。但缺点也是非常明显的：一旦硬件发生改动，应用往往也得随之改动，很难做到让这个应用在不同的硬件平台都能运行。一种可能的方法是：实现一套用于硬件资源管理与抽象的库，具有相对稳定的接口，并将其作为应用的一部分。这样当硬件发生改动时，只需要修改这个库就行了，不需要修改应用。事实上，有一些对性能和时延要求极高的场景就是通过类似的方法实现的，包括本书后面将介绍的基于外核（Exokernel）架构的库操作系统（LibOS）与虚拟化容器等方法。在这种情况下，与其说不需要操作系统，不如说操作系统的功能以库的形式存在——这也是“库操作系统”这个名字的由来。

1.3 操作系统简史

1.3.1 GM-NAA I/O：第一个（批处理）操作系统

1946 年 2 月 14 日，世界上公认的第一台通用计算机 ENIAC 在美国宾夕法尼亚大学诞生。在这之后，如何管理计算机上的程序也逐步成为一个非常重要的问题。早期的计算机采用纸带的方式记录要计算的任务，并通过打孔的纸带或磁带接收输出的结果。这样就需要一个专门的操作员值守在计算机旁边，并且操作的效率也不高。因此，人们开始关注如何自动化这一过程。

1956 年，Robert L. Patrick 和 Owen Mock 在 IBM 704 上实现了第一个实际使用的操作系统 GM-NAA I/O^[1]，即通用汽车公司和北美航空的输入 / 输出系统（General

Motors and North American Aviation Input/Output Systems)^①。从名字上可以看出，GM-NAA I/O 实现的主要是对输入与输出的自动化管理：操作员只需要将相关的任务交给 GM-NAA I/O，计算机就会一直工作到所有任务执行结束。因此，GM-NAA I/O 实现的也就是批处理（Batch Processing）。具备批处理特性的一系列操作系统也被称为批处理操作系统。

1.3.2 OS/360：从专用走向通用

1964 年，美国 IBM 公司发布了名为 IBM System/360 的大型机。IBM 为此前后投入了 50 亿美元，总共有 6 万多名员工参与了该系统的研制，至 1989 年，该系统占了当时全球所有公司计算机存量的一半（价值约为 1 300 亿美元）^②。它不仅是一款商业上非常成功的大型机，而且在计算机历史上实现了两个突破：

- 通过定义对应的指令集架构（Instruction Set Architecture, ISA），将计算机的架构与实现进行解耦。ISA 类似于硬件和软件之间的“协议”，约定了机器指令的格式、作用以及硬件状态等，上层软件用这套协议来编写，下层硬件负责实现这套协议，因此在 ISA 不变的情况下，底层硬件可以完成多次迭代更新而不影响上层软件；并且，还允许使用者通过自定义架构扩展外设等计算功能，这也是 20 世纪 80 年代 IBM PC（Personal Computer）兼容机的基础。
- 由于指令集架构与具体实现的分离，IBM System/360 的成功也标志着操作系统从面向每种计算机的定制——每个计算机都要实现一套操作系统，转变为与计算机底层硬件实现解耦。自此之后，操作系统开始进入通用操作系统时代。

值得一提的是，IBM System/360 的系统总架构师是 Gene Amdahl，他在后来提出了并行计算中著名的阿姆达尔定律（Amdahl's law）^③；项目经理是 IBM 的系统部主任 Frederick P. Brooks, Jr.，他带领团队研发了 IBM System/360 的操作系统 OS/360。Brooks 教授后来去了北卡罗来纳大学教堂山分校，并根据他在这个项目中的开发与管理经验，撰写了著名的《人月神话：软件项目管理之道》（*The Mythical Man-Month: Essays on Software Engineering*）^[2]，开创了软件工程这门学科。Brooks 教授也因为在计算机体系结构、操作系统和软件工程领域的开创性贡献获得了 1999 年的图灵奖。

1.3.3 Multics/UNIX/Linux：分时与多任务

UNIX 起源于 1964 年开始的由通用电气（General Electric，简称 GE）和麻省理工学院（MIT）联合发起的 Multics 项目^[3]。Multics 是 Multiplexed Information and Computing

① 参见 <http://www.softwarepreservation.org/projects/os/gm.html>。

② IBM 官方数据，参见 <https://www-31.ibm.com/ibm/cn/ibm100/icons/system360/index.shtml>。

③ 阿姆达尔定律刻画了并行计算中的加速比与并行程序中的并行部分占比（ p ）的关系，即一个并行程序最大的加速比是 $1/((1-p)+p/s)$ ，其中 s 是并行部分的加速比。

Service 的简写，目标是设计与实现一套多用户、多任务、多层次的操作系统。Multics 使用了分时（Time-sharing）的概念，并且首次将文件与内存进行分离，也提出和实现了文件系统、动态链接（Dynamic Linking）、CPU/内存/磁盘等硬件热替换（Online Reconfiguration）、特权级分层（Protection Rings）、命令处理器（也就是后来的 shell）等开创性概念。Multics 是最早一批用高级语言编写的操作系统之一，其所使用的高级语言就是 Multics PL/I 语言。

贝尔实验室（Bell Labs）于 1964 年加入 Multics 项目。由于 Multics 的复杂性导致开发进度缓慢，1969 年贝尔实验室退出了 Multics 项目。不久，曾参与过 Multics 项目的两位贝尔实验室的工程师，也就是 Ken Thompson 和 Dennis Ritchie，开发了 UNIX 系统^[4]。UNIX 系统的原名是 Unics，意指与 Multics 相比较是 Uniplexed 而不是 Multiplexed——尽管后来 UNIX 也支持多用户、多任务时分复用等 Multics 原先的设计目标。UNIX 中引入了 shell，通过命令行解释器（Command-line Interpreter）执行不同的计算任务，并支持通过管道（Pipe）等进程间通信方式将不同的计算任务连接起来。

UNIX 在第 4 版以前都是基于汇编语言构建的。由于汇编语言的移植性不佳，Dennis Ritchie 于 1972 年到 1973 年设计与实现了 C 语言，并在 1973 年用 C 语言重写了 UNIX，也就是 UNIX 的第 4 版。后来，UNIX 和 C 语言渐渐成为大家熟知的操作系统与编程语言，并被普及到各种型号的计算机上。Ken Thompson 和 Dennis Ritchie 也凭借 UNIX 和 C 语言而获得了 1983 年的图灵奖。

由于 UNIX 系统版权复杂且收费，人们一直难以拥有一款公开且允许自由使用的操作系统。虽然荷兰自由大学的 Andrew S. Tanenbaum 教授在 1987 年开源了用于教学的 MINIX 操作系统，但是由于商用版权不友好，MINIX 并不能被自由使用。在学习了 MINIX 的设计与实现细节后，来自芬兰的年轻程序员 Linus Torvalds 于 1991 年发布了 Linux 操作系统，此系统已成为目前世界上最成功、使用范围最广的开源操作系统。

图 1.2 给出了分时与多任务操作系统的演进过程。



图 1.2 分时与多任务操作系统的演进过程

1.3.4 macOS/Windows：以人为本的人机交互

在包括 UNIX 在内的早期操作系统中，用户主要通过使用 shell 命令行与操作系统进行交互。这种交互方式对于专业程序员而言没有问题，但是对于普通用户却是一个非常大的挑战。

最早的图形用户界面（Graphical User Interface，GUI）操作系统出现在 1973 年 Xerox PARC^①的 Alto 计算机中，其 GUI 如图 1.3^②所示。Xerox Alto 是第一个引入桌面（Desktop）概念的计算机，也是世界上第一台个人计算机。它的发明人 Chuck Thacker 也因为设计与实现了第一台现代意义上的个人计算机 Xerox Alto 而成为 2009 年图灵奖得主。

1979 年，苹果公司的创始人 Steve Jobs 访问了 Xerox PARC，意识到 GUI 的重要性，开始开发面向图形化的计算机与操作系统，并且在 1983 年发布了 Apple Lisa 个人桌面计算机，也就是后来著名的 Macintosh 系列个人计算机。

在苹果公司的个人计算机大获成功后，微软公司的创始人 Bill Gates 也意识到图形用户界面的重要性，随后投入重兵开发了基于图形界面的操作系统 Windows。Windows 1.0 于 1985 年发布并且大获成功。据说 Steve Jobs 当时曾当面斥责 Bill Gates 偷盗与山寨了他的技术，而 Bill Gates 则给出了一段非常经典的回答：“我们都有个有钱的邻居，叫施乐，我闯进他们家准备偷电视机的时候，发现你已经把它盗走了。”^③这也开启了苹果与微软两家伟大的公司长达十多年的法律纠纷。

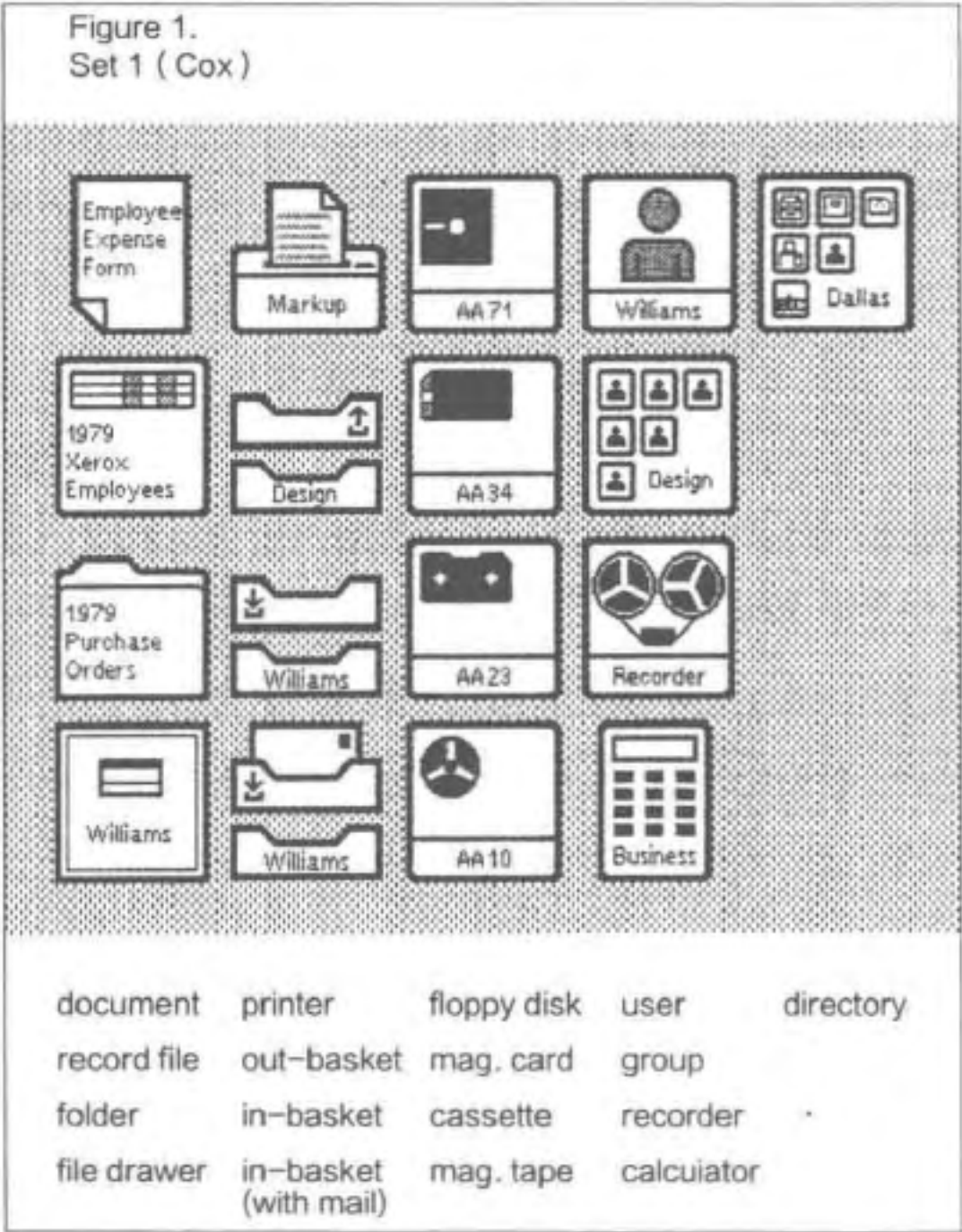


图 1.3 Xerox Alto 的图形化界面

图 1.4 给出了图形化接口操作系统的历史进程。

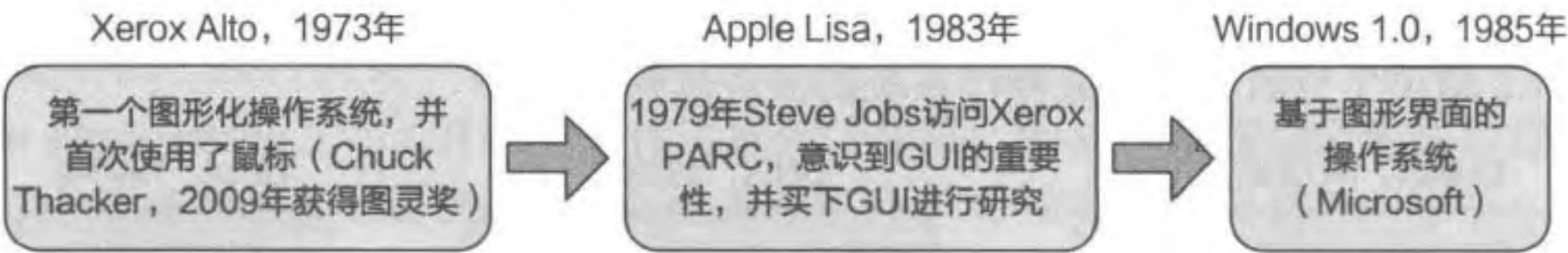


图 1.4 图形化接口操作系统的历史进程

1.3.5 iOS/Android：移动互联网时代的操作系统

尽管 PC 时代的操作系统极大地简化了普通用户使用计算机的难度，但 PC 和笔记本

① 施乐帕罗奥多研究中心，现为 PARC，即 Palo Alto Research Center（帕罗奥多研究中心）的缩写。
② 图片源自 https://upload.wikimedia.org/wikipedia/commons/0/01/Icons_on_the_Star.jpg，基于 CC BY-SA 协议使用。
③ 参见 <https://www.businessinsider.com/bill-gates-answers-reddit-question-about-copying-steve-jobs-2017-3>。

电脑的重量与尺寸依然限制了人们使用计算机的自由度，基于键盘、鼠标（含触点杆和触摸板）的交互方式也阻碍了计算机的进一步普及。与此同时，随着移动通信网络技术的不断发展，高速无线数据传输的支持使得无处不在的互联网访问成为可能，这开启了移动互联网时代。

2007 年，Steve Jobs 发布了第一代 iPhone，同样标志着 iOS 操作系统的诞生。与之前的智能手机不同的是，iPhone 采用触摸屏的交互方式，从而支持更加自然的人机交互。此外，iOS 也在应用与应用模式上具有显著的创新，新的 AppStore 一方面提供了移动应用分发与销售的新模式，另一方面也为移动生态的构建提供了新的基础。

图 1.5 显示了 iOS 与 macOS 的架构分层及异同。iOS 的设计在很大程度上复用了 macOS 的功能，包括 XNU 操作系统内核（由 Mach 微内核和 BSD 内核叠加构成的混合内核），以及一些核心 OS 服务（Core OS Services），并将 macOS 的应用程序框架（Cocoa）针对 iPhone 等触屏设备进行了重新设计与优化，形成了新的面向智能手机的应用程序框架 Cocoa Touch。值得一提的是，苹果公司面向其他设备的操作系统如 iPadOS、tvOS 与 watchOS，也是根据设备的特点从 iOS 衍生出来的。这些不同的操作系统都通过 iCloud 提供云服务，例如账号管理、存储、应用分发、支付等，从而带来跨多设备的一致性体验。

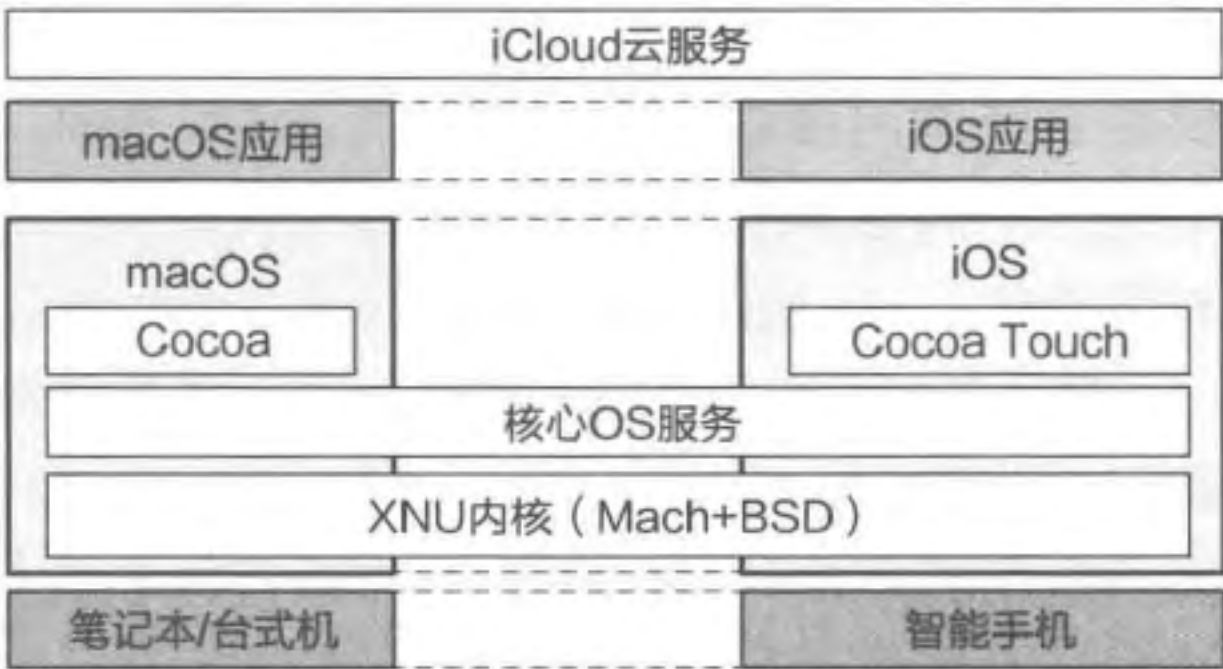


图 1.5 iOS 与 macOS 的架构分层及异同

移动互联网时代的来临也促使谷歌公司投入移动操作系统的研发，从而抢占新时代的入口（如移动搜索和移动广告）。谷歌公司在 2005 年收购了 Andy Rubin 主导设计的 Android 操作系统，并在此基础上发布了 Android 1.0 操作系统。在图 1.1 中，我们已经展示了 Android 操作系统的基本架构。不同于 iOS 采用核心代码闭源、生态强管控的方式，Android 采用开源的方式来构建生态。2007 年 11 月，谷歌公司联合 34 家芯片制造商、网络运营商、手机设备生产商与应用开发者成立开放手机联盟（Open Handset Alliance, OHA），从而使 Android 逐步在移动互联网时代的操作系统中占据了主流地位。

此外，由于移动应用开发越来越多地需要使用云服务提供的能力，诞生了移动后端即服务（mobile Backend as a Service, mBaaS）模式：操作系统将应用开发所需要的一些通用的后端功能（如账号管理、推送通知、社交服务等）汇聚成一个服务集合，以 SDK 和 API 的形式提供给移动应用开发者。例如，苹果公司提供了 CloudKit 以方便应用开发，谷歌公司也基于其收购的 Firebase 构建了谷歌移动服务（Google Mobile Services, GMS）。由于 CloudKit 和 GMS 沉淀了很多应用的共性功能并提供了云服务的入口，它们也成为重要的生态入口与控制点。

此外，由于移动应用开发越来越多地需要使用云服务提供的能力，诞生了移动后端即服务（mobile Backend as a Service, mBaaS）模式：操作系统将应用开发所需要的一些通用的后端功能（如账号管理、推送通知、社交服务等）汇聚成一个服务集合，以 SDK 和 API 的形式提供给移动应用开发者。例如，苹果公司提供了 CloudKit 以方便应用开发，谷歌公司也基于其收购的 Firebase 构建了谷歌移动服务（Google Mobile Services, GMS）。由于 CloudKit 和 GMS 沉淀了很多应用的共性功能并提供了云服务的入口，它们也成为重要的生态入口与控制点。

1.4 操作系统接口

操作系统在几十年的演进过程中形成了一些相对稳定的接口，这些接口又沉淀到不同的层次。越是下层的接口，数量相对更少，变化相对不频繁；越是上层的接口，数量相对更多，变化相对更频繁。图 1.6 展示了不同层次的操作系统接口，主要包括系统调用接口、POSIX 接口、领域应用接口。

系统调用接口。应用程序通过操作系统内核提供的接口向内核申请服务，这些接口通常称为系统调用接口。不同的操作系统提供的系统调用接口往往各不相同，同一操作系统的不同版本所提供的系统调用接口也会有所变化。



图 1.6 不同层次的操作系统接口

图 1.7 以 hello 程序中的 `printf` 为例展示了系统调用的执行过程。首先，`printf` 调用标准库 `libc` 中的 `write` 函数。`libc` 在准备好相关的参数后，执行 `svc` 指令^①使控制流从应用程序的代码转移到操作系统内核的代码，即运行主体由应用程序切换为操作系统内核。内核的处理函数根据系统调用传入的第一个参数，识别出该调用需要执行操作系统内核提供的 `sys_write` 函数，从而通过系统调用表找到并调用该函数。从上述例子可以看出，系统调用是用户态应用向操作系统内核请求服务的方法。

POSIX 接口。由于每个操作系统提供的系统调用各不相同，为了实现同一应用程序在不同操作系统上的可移植性，逐渐形成了关于操作系统接口的一些标准，POSIX 是其中应用最广泛的一个。POSIX 是 Portable Operating System Interface for uniX 的简写，即可移植操作系统接口，X 表明其是对 UNIX API 的传承。在 POSIX 被提出之前，世界上存在很多不同的 UNIX 操作系统，如 FreeBSD、UNIX System V、Solaris、NetBSD 等。这些接口各异的操作系统对应用程序开发人员造成了比较大的困扰。为了使应用程序能够运行在不同的 UNIX 操作系统之上，IEEE 于 20 世纪 80 年代定义了一套标准的操作系统 API，其正式名称为 IEEE Std 1003，国际标准名称为 ISO/IEC 9945。POSIX 这个名称是由开源软件先驱 Richard Stallman 应 IEEE 的请求而取的一个易于记忆的名称。POSIX 标准通常通过 C 语言库 (C library, `libc`) 在用户地址空间实现，常见的 `libc` 包括 `glibc`、`musl`、`eglibc` 等，Android 也实现了一个名为 `bionic` 的 `libc`。通常而言，应用程序只需要调用 `libc` 提供的接口就可以实现对操作系统功能的调用，这样一方面可支持应用在类 UNIX 系统 (包括 Linux) 上的可移植性，另一方面也使新的操作系统可以通过移植 `libc` 支持现有的应用生态。

① `svc` 指令是一条 ARM 指令，表示系统调用 (Supervisor Call)。